Analyse d'identité d'auteurs et détection de plagiat

CHEDJARI EL MEUR Hadjer
NANMEDIGOU Yendoumban
Université de Caen Normandie

Projet 2

ADDAD Youva



Sommaire	02
Introduction	03
Objectifs du Projet	04
Pré-analyse des données	05
Methodologie	06
Featurizers	07
Classifiers	10
Experimentations	21
Conclusion	34
Répartition des tâches	35



Sommaire

Introduction

01

Contexte

Avec la numérisation croissante, la diffusion de contenus académiques et professionnels est facilitée, mais cela augmente aussi les risques de plagiat.

02

Pourquoi ?

Le plagiat remet en cause la crédibilité des travaux et nécessite des outils fiables pour garantir l'authenticité des documents.

03

Besoin de Recherche

Face à ces enjeux, il est essentiel de développer des solutions automatisées capables de détecter efficacement les similarités entre textes.

Objectifs du projet

Objectif #1

Développer une solution de détection de plagiat différents algorithmes.

Objectif #2

Comparer nos solutions à des bibliothèques existantes telles que NLTK et Scikit-learn.

Objectif #3

Évaluer la performance des modèles à l'aide de métriques adaptées.



PRÉ-ANALYSE DES DONNÉES

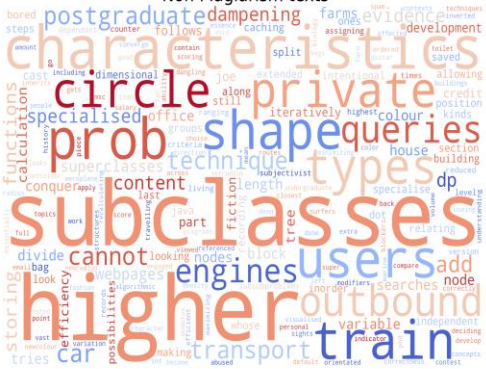
- Un corpus expréssement créé pour la détection de plagiat
- Une similiraté des textes remarquable
- Des wordclouds néanmoins parlant
- Quelle frontière établir entre les 5 classes du corpus ?



All texts



Non Plagiarism texts



Plagiarism texts



Methodologie



Tokenizer

Divise le document en entrée en tokens après la mise en minuscules et la suppression des stop words.



Featurizer

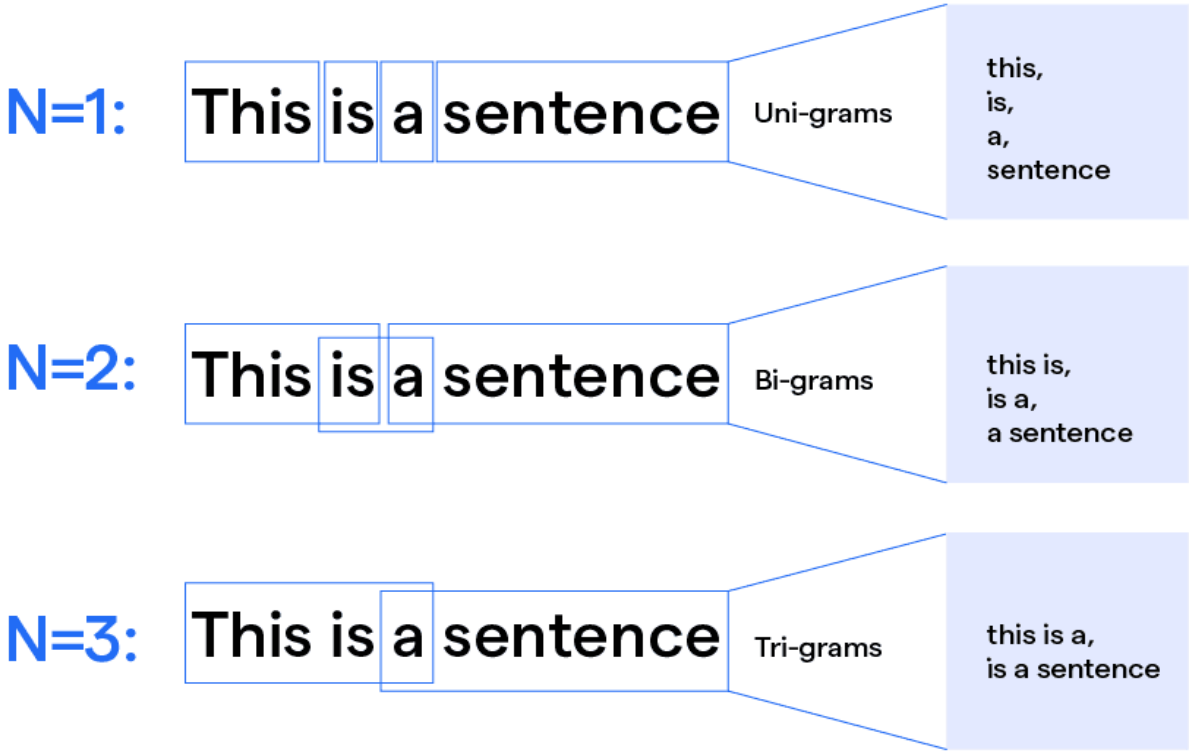
Transforme les tokens en représentations numériques exploitables par les algorithmes d'apprentissage automatique.



Classifier

Utilise ces représentations pour s’entraîner et prédire si un document est un "plagiat" ou "non plagiat".

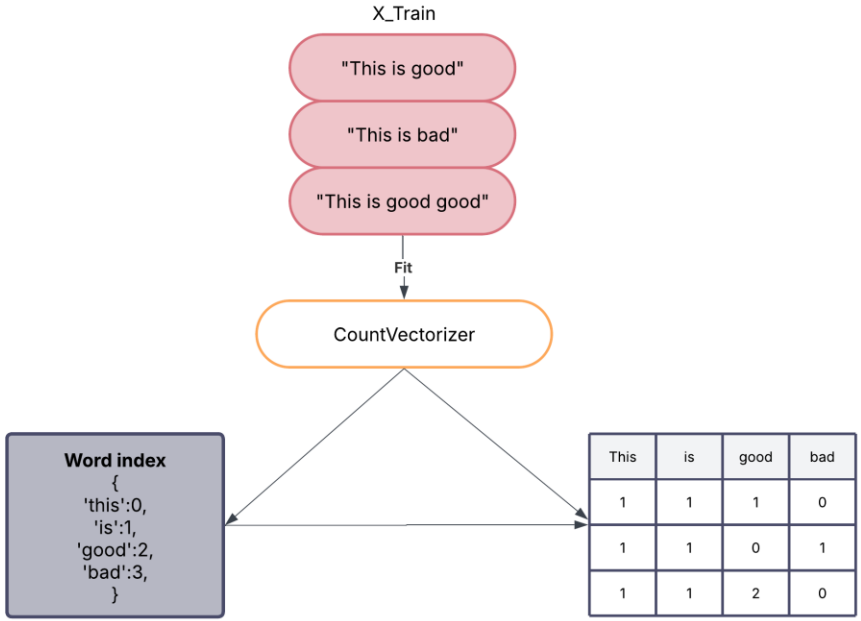
N-grams


N=1:
This is a sentence
Uni-grams
this,
is,
a,
sentence
N=2:
This is a sentence
Bi-grams
this is,
is a,
a sentence
N=3:
This is a sentence
Tri-grams
this is a,
is a sentence

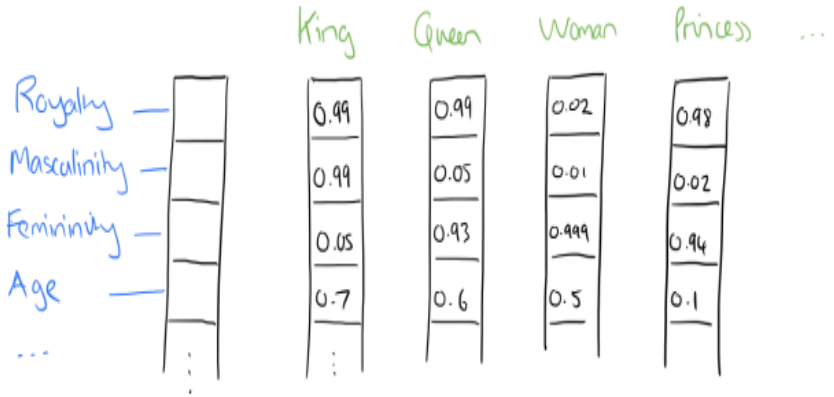
Count Vectorizer

- **Indexation**: Associe un index unique à chaque mot du vocabulaire.
- **Comptage**: Calcule le nombre d'occurrences de chaque mot dans les documents d'entraînement.
- **Représentation**: Génère une matrice où :
  - Chaque ligne représente un document.
  - Chaque colonne correspond à un mot du vocabulaire.

Count(w,d) = nombre d'occurrences du mot "w" dans le document "d".



TF-IDF



- Technique d'extraction de caractéristiques textuelles.
- Pondère les mots en fonction de :
  - Leur fréquence dans un document.
  - Leur rareté dans l'ensemble du corpus.
- Objectif :
  - Donner plus de poids aux mots significatifs.
  - Réduire l'influence des mots fréquents mais peu informatifs (ex : "le", "et", "de").

$$TF(t, d) = \frac{\text{Nombre d'occurrences de t dans d}}{\text{Total de mots dans d}} \qquad IDF(t) = \log\left(\frac{N}{1 + DF(t)}\right)$$

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

Classifiers



Basé sur la probabilité #1

Modèles qui utilisent des principes statistiques pour estimer la probabilité d'appartenance à une classe.

Basé sur les fonctions linéaires #2

Modèles qui prennent des décisions en traçant une frontière linéaire entre les classes.

Basé sur les arbres de décision #3

Modèles qui utilisent une structure arborescente où chaque nœud représente une règle de décision.

Basé sur les ensemble de modèles #4

Méthodes qui combinent plusieurs modèles faibles pour améliorer la précision globale.

La probabilité

Naive Bayes

Les fonctions linéaires

Logistic Regression

Perceptron

Les arbres de décision

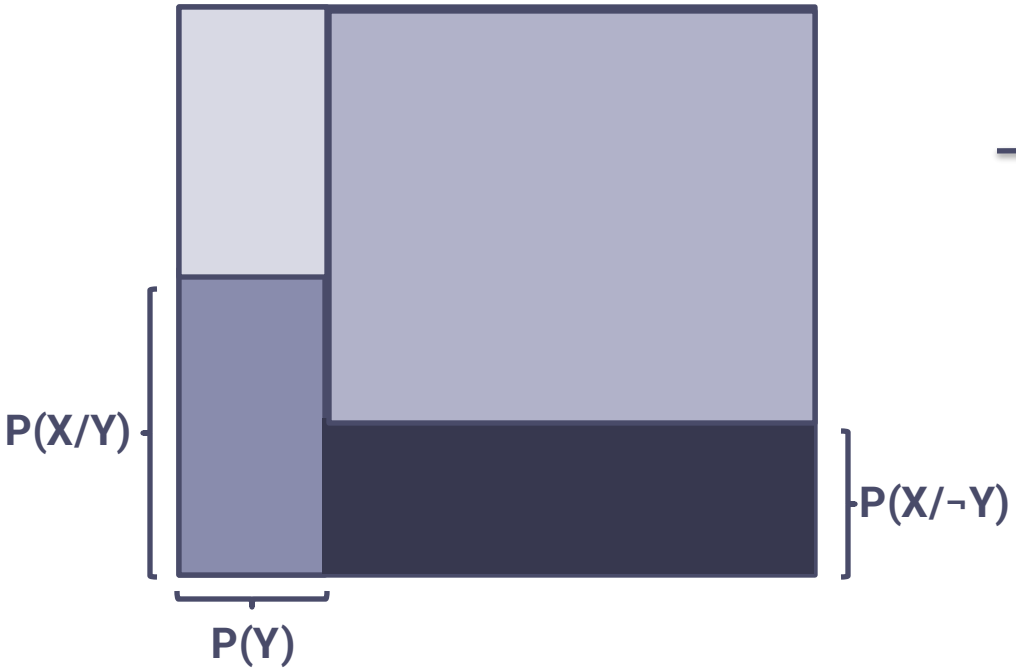
Desicion Tree

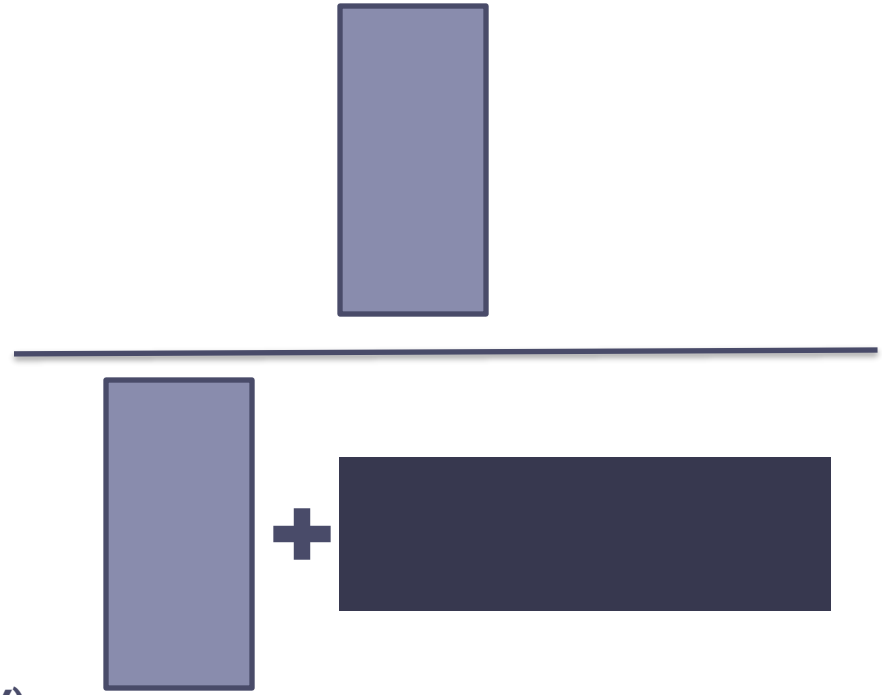
L'ensemble de modèles

Random Forest

AdaBoost

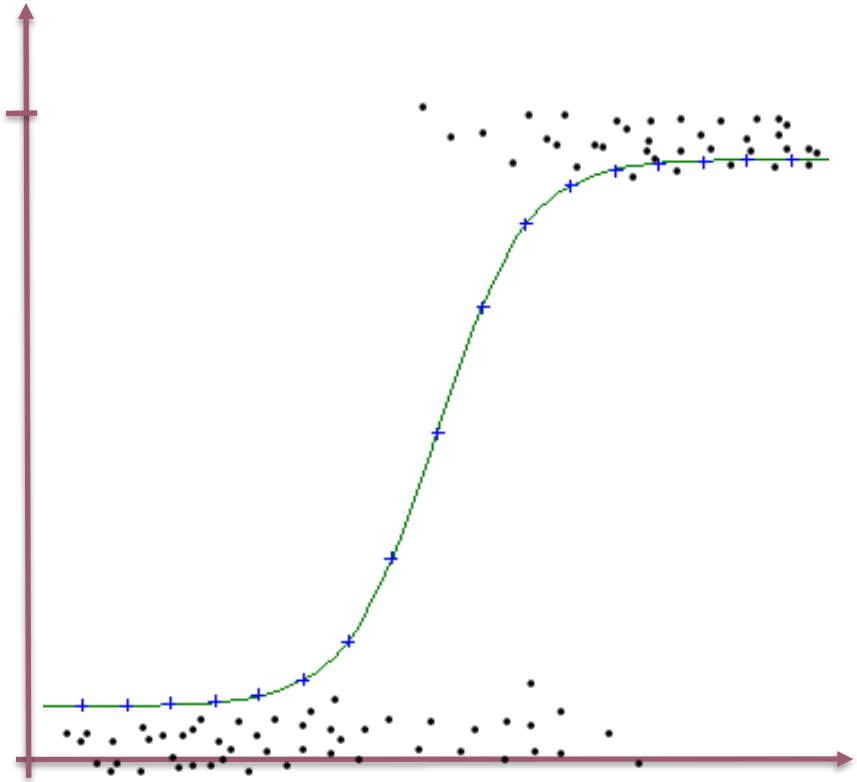
Naive Bayes





$$P(y|x) = \frac{P(x|y) \cdot P(y)}{P(x)}$$

Logistic Regression



$$\sigma(x) = \frac{1}{1 + e^{-(wx+b)}}$$

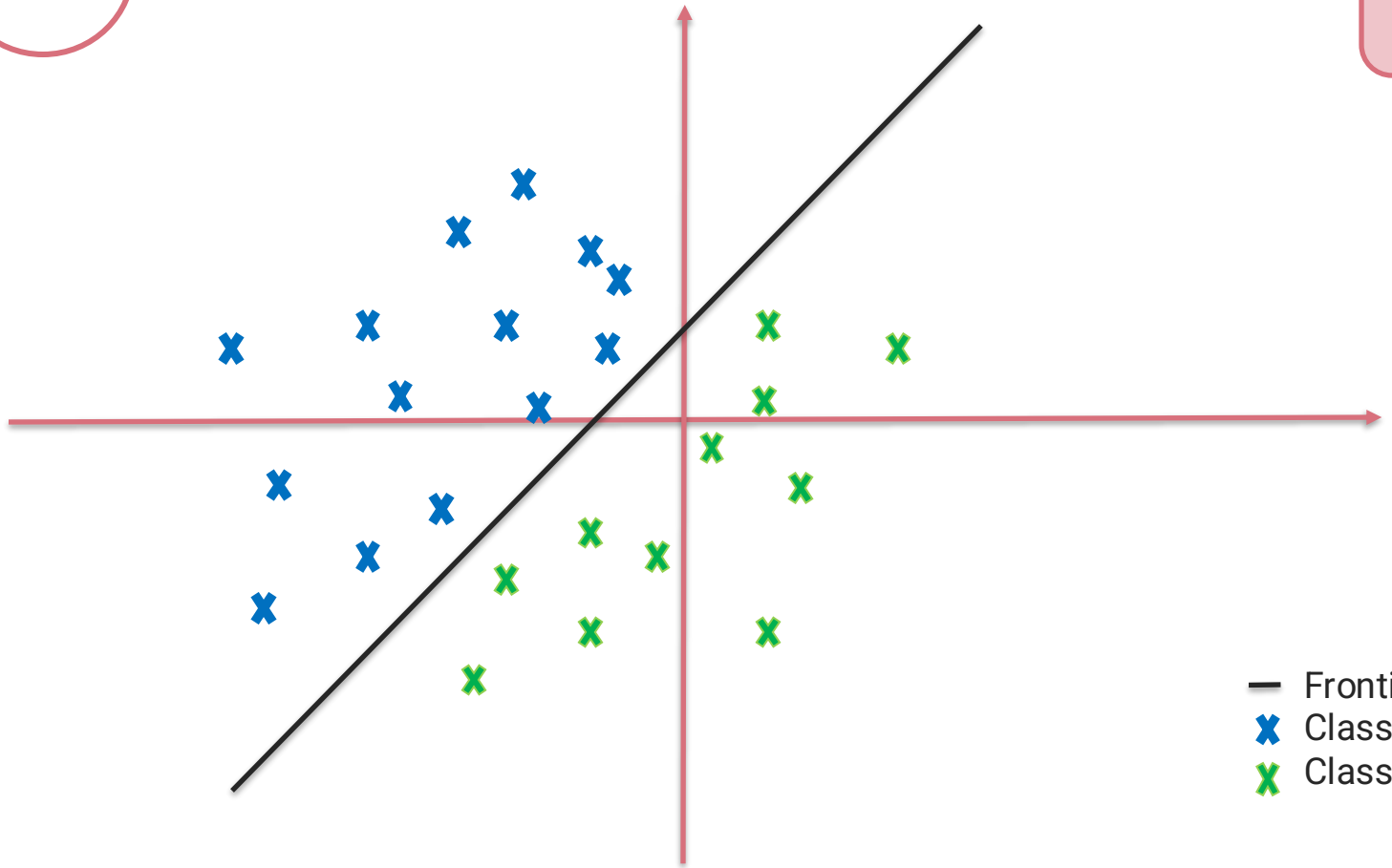
- $\sigma(x) \in [0,1] \rightarrow$ probabilité

$$y = \begin{cases} 1, \text{si } \sigma(x) \geq 0.50 \\ 0, \text{si } \sigma(x) < 0.50 \end{cases}$$

- $y \in \{0,1\} \rightarrow$ réponse binaire

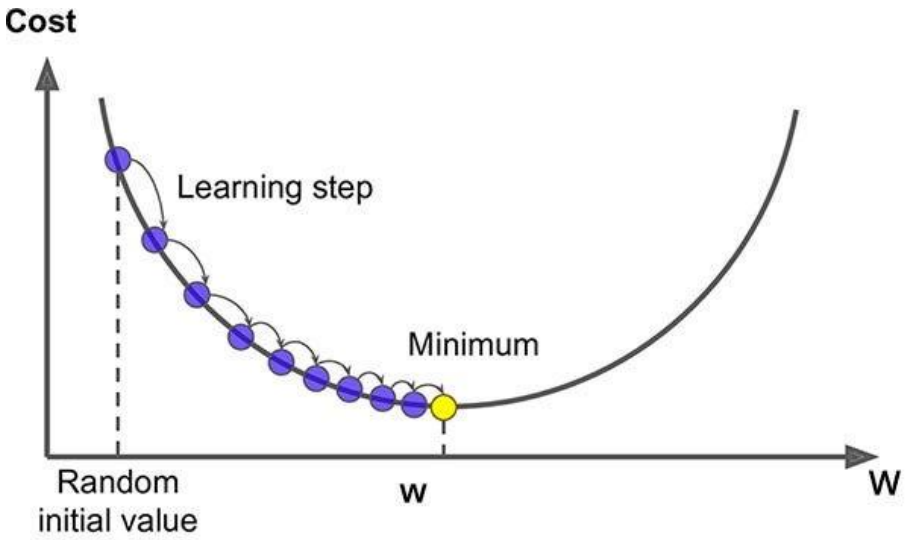
Perceptron

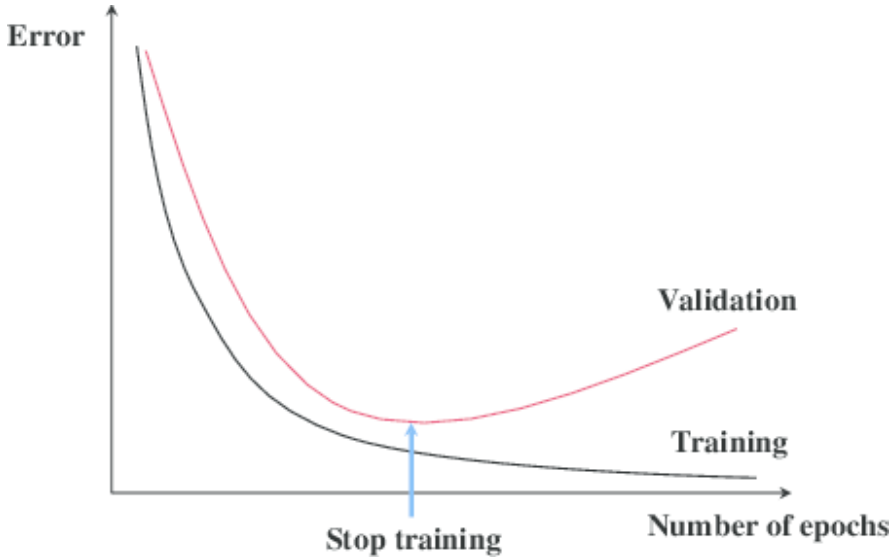
$y = signe(wx + b)$



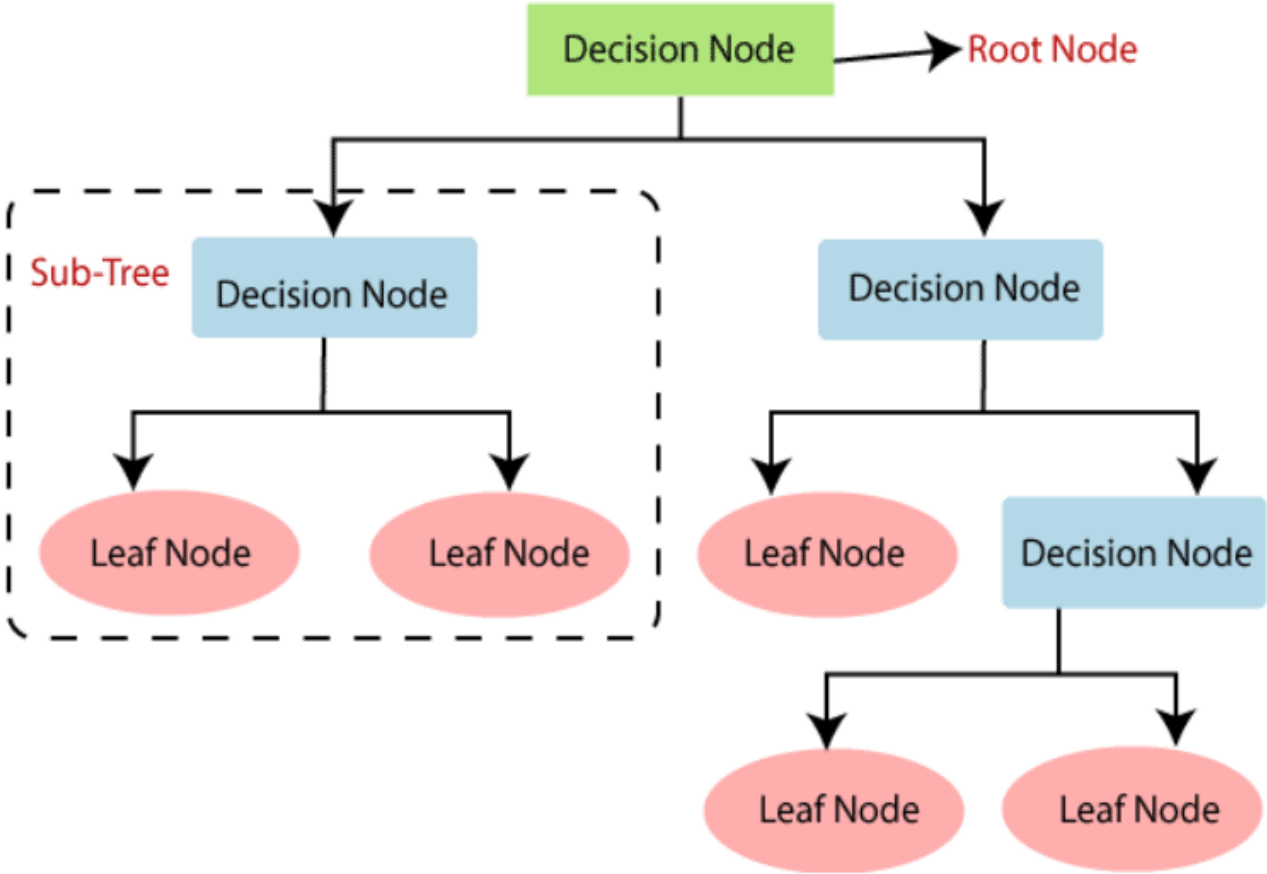
Frontière de décision (z=0)
Classe 1 Plagiat (z=1)
Classe 0 Non Plagiat (z=-1)

Descente de Gradient et Arrêt Précoce





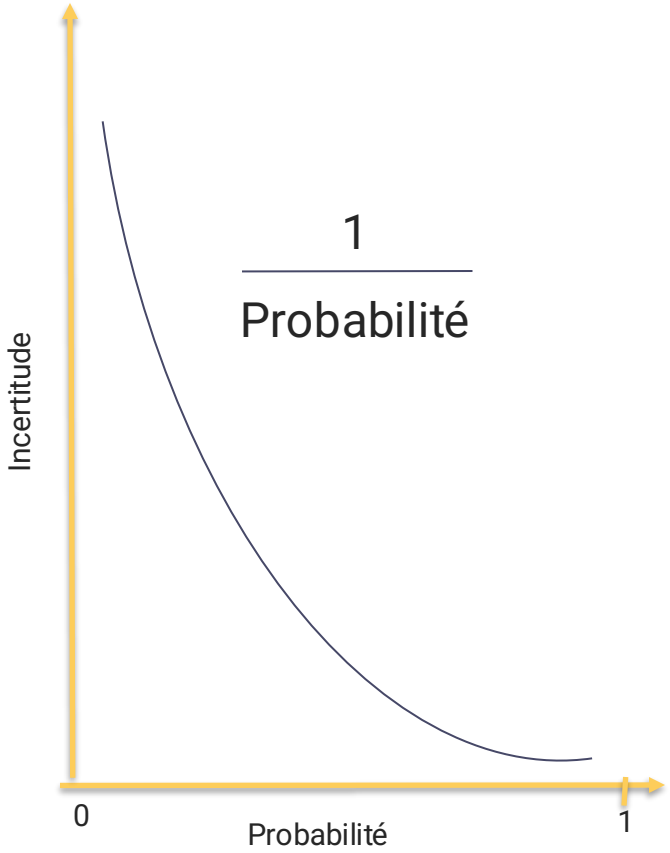
Decision Tree



Decision Tree

#Entropy

$$\text{Entropy} = \Sigma \log\left(\frac{1}{p(x)}\right) p(x)$$



→ $\Sigma p(x)[\ \log(1) - \log(p(x))\ ]$

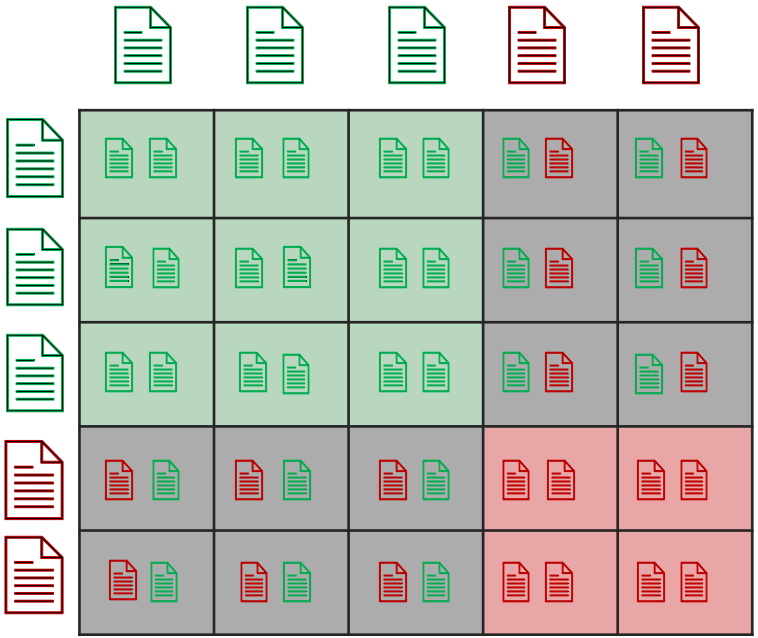
→ $\Sigma p(x)[\ 0 - \log(p(x))\ ]$

→ $-\Sigma p(x) \log(p(x))$

Decision Tree

#Gini Impurity

Gini = 1 - ΣPi^2

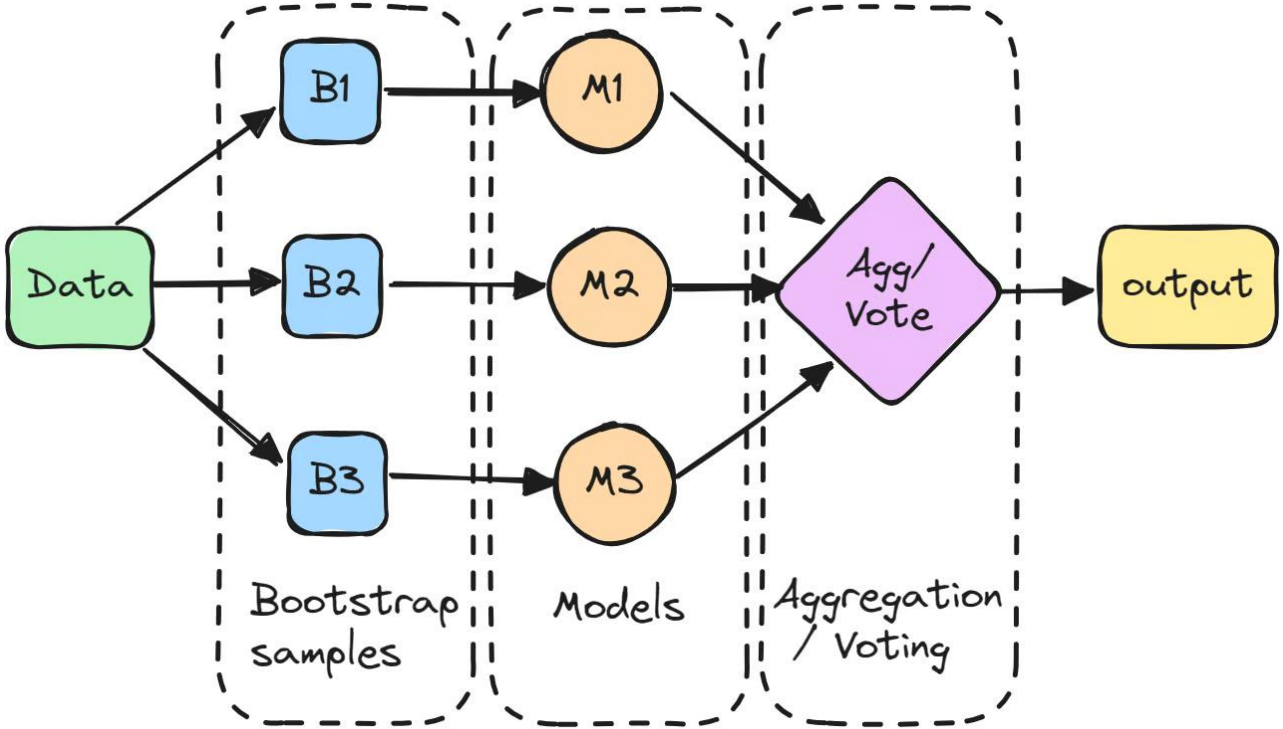




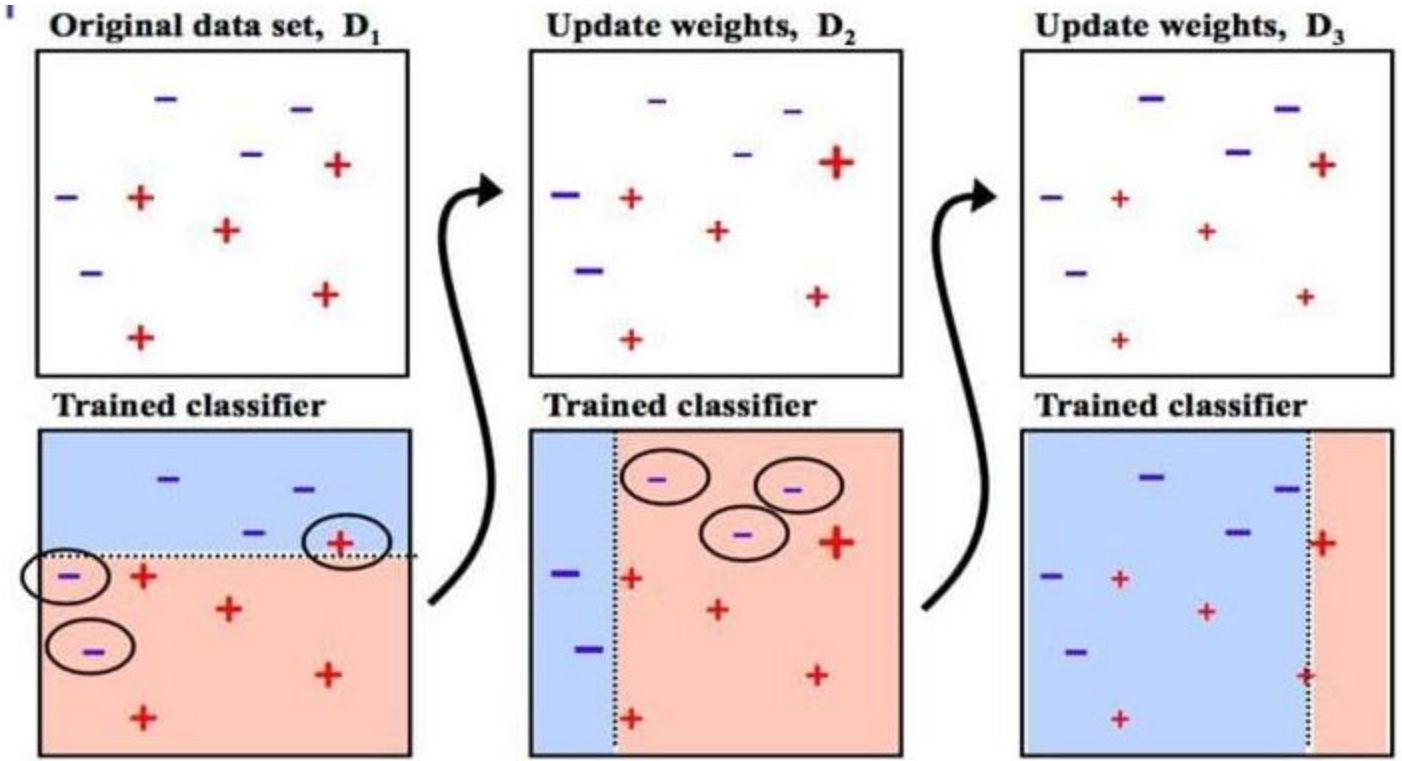
P(diff) = P(All) - P(égal)

= 1 - P(both non plagiat) - P(both plagiat)

Bagging (Random Forest)



Ada Boost


Original data set, D_1
Update weights, D_2
Update weights, D_3
Trained classifier
Trained classifier
Trained classifier

EXPÉRIMENTATIONS

- Evaluer pour mieux comprendre et améliorer
- Plusieurs métriques évaluées
- Plusieurs angles d'expérimentations
- Comparer aux bibliothèque existantes



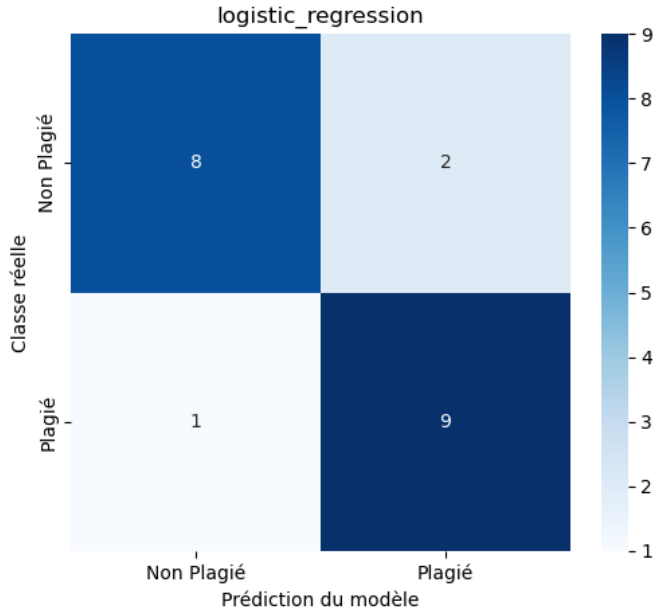
MÉTRIQUES ÉVALUÉES

- Nécessité d'évaluer plusieurs métriques
- Des vrai positifs, vrais négatifs, faux positifs et faux négatifs
- L'accuracy et la balanced accuracy
- La précision, le rappel et le f1 score



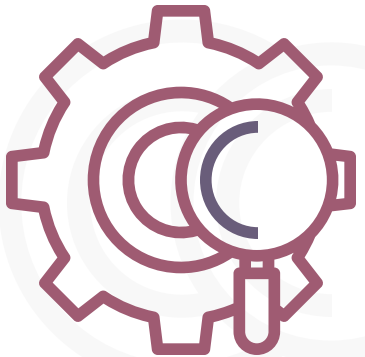
MÉTRIQUES ÉVALUÉES

- Matrices de confusion

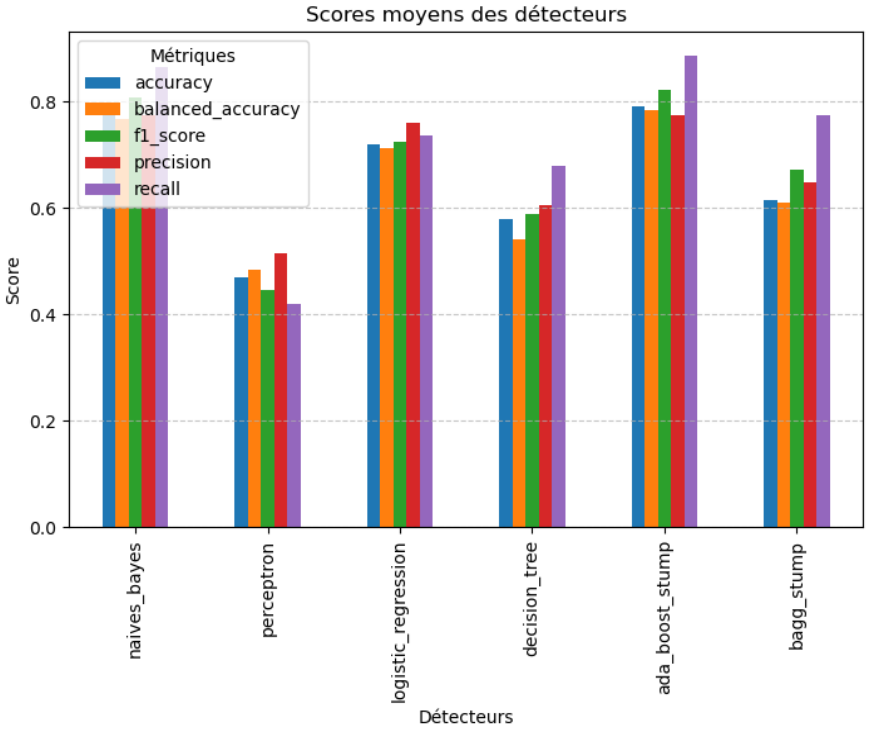


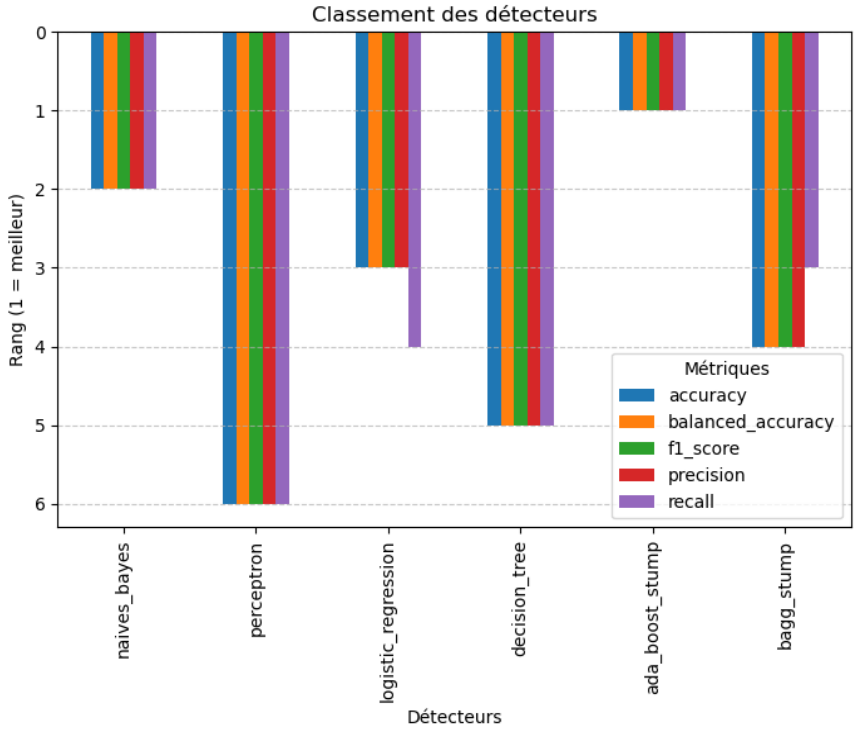
CLASSIFIERS LES PLUS PERFORMANTS ?

- Une place centrale dans le modèle d'apprentissage
- Naïve Bayes et Ada Boost en tête
- Logistic Regression et Random Forest intéressants
- Perceptron et Decision Trees moins adaptés



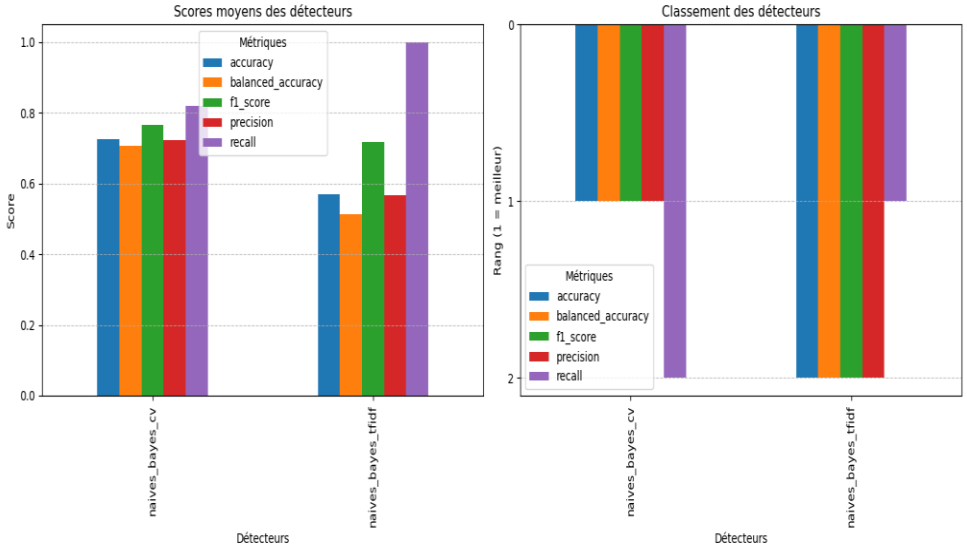
CLASSIFIERS LES PLUS PERFORMANTS ?

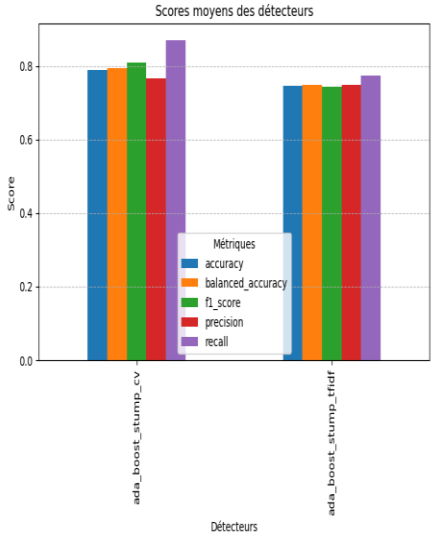


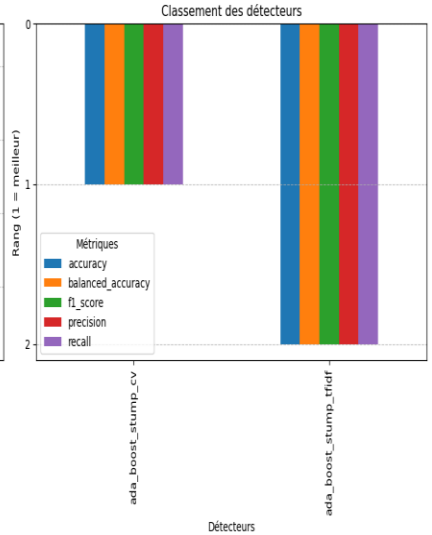


ÉTUDE DES FEATURIZERS

- Choix du featurizer

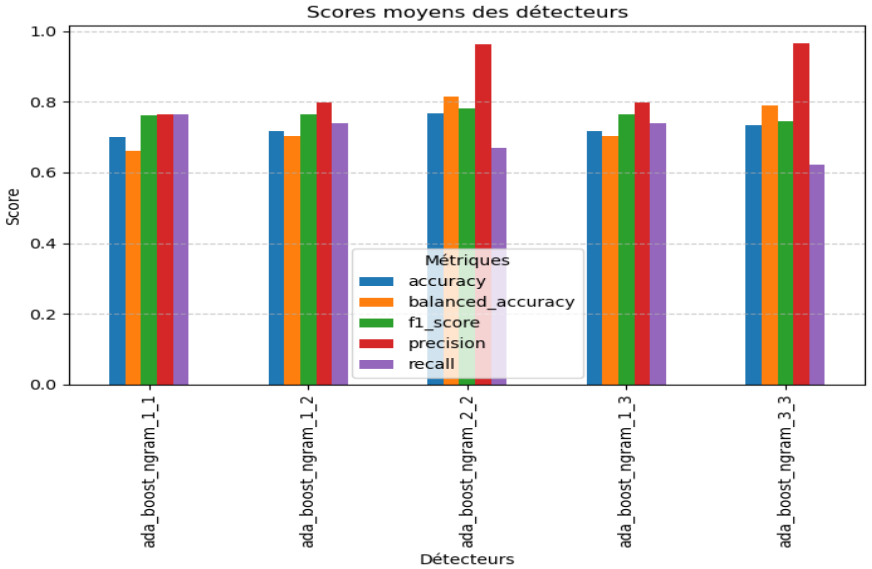


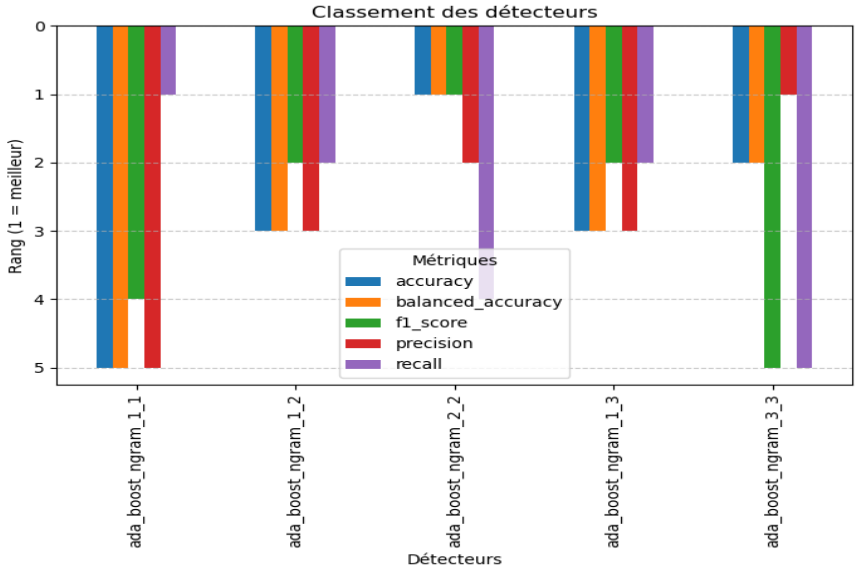




ÉTUDE DES FEATURIZERS

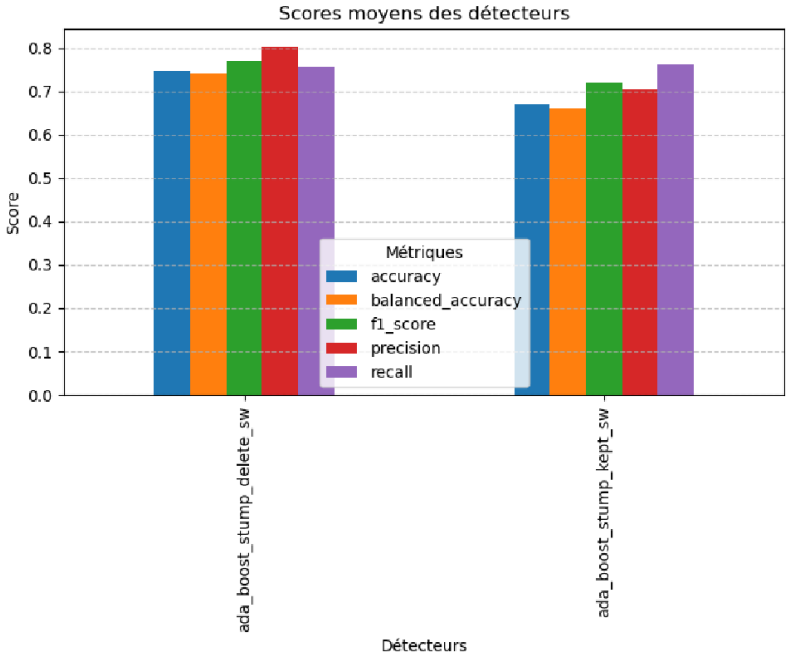
- ngrams

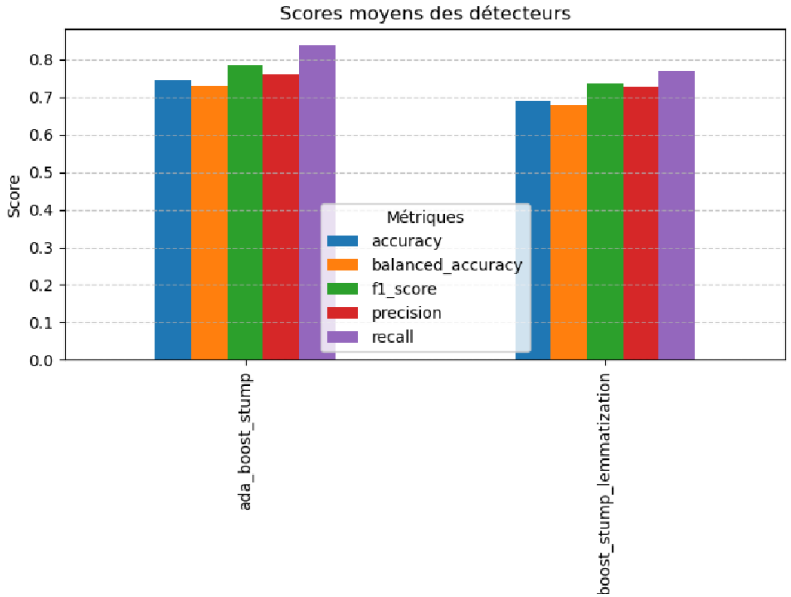




PRÉPROCESSING NÉCESSAIRE

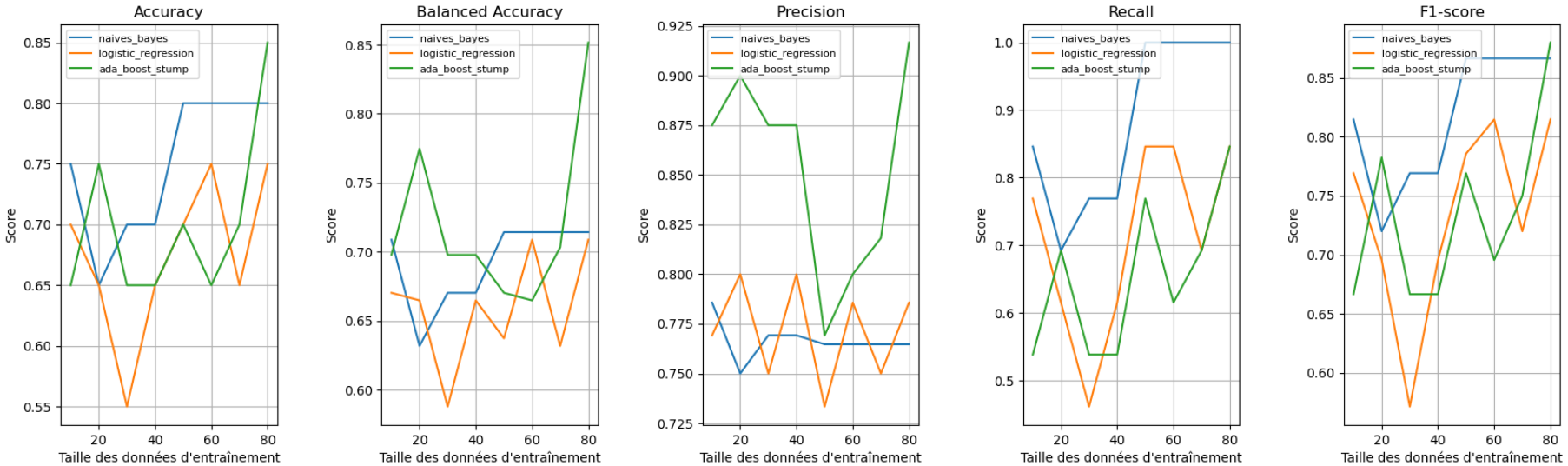
- Les stopwords, trop présents : non discriminants
- Lemmatisation : un impact remarquable





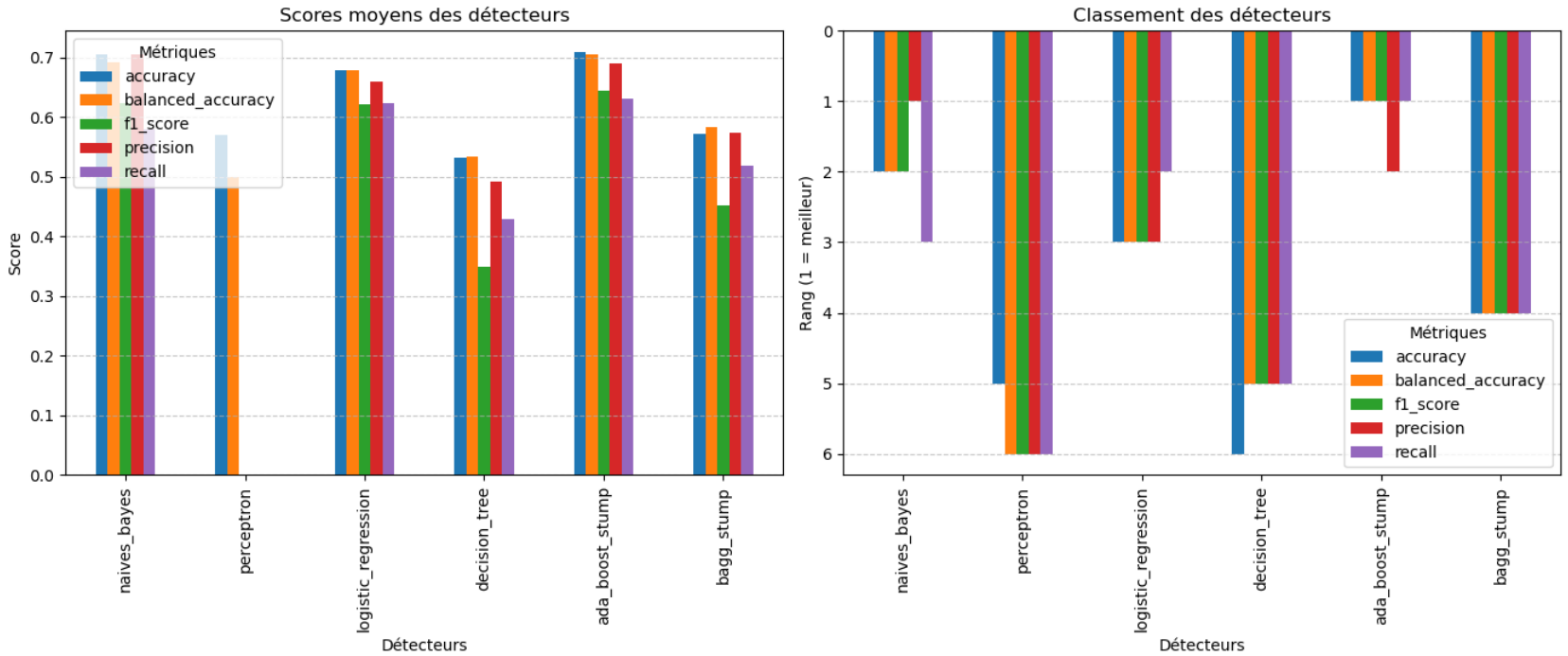
IMPACT DE L'APPRENTISSAGE

- Une intuition confirmée
- Une croissance des performances non stricte néanmoins
- Pas de piège de surapprentissage détecté



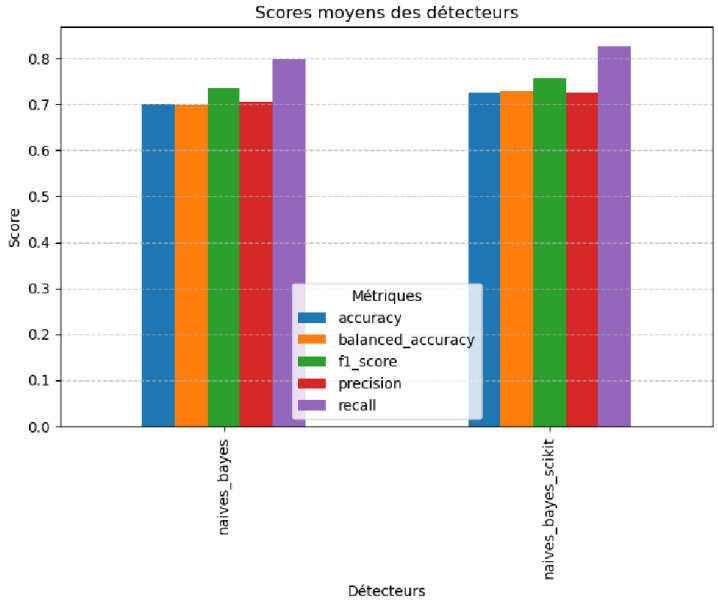
UNE CLASSE PLUS FACILE À DETECTER ?

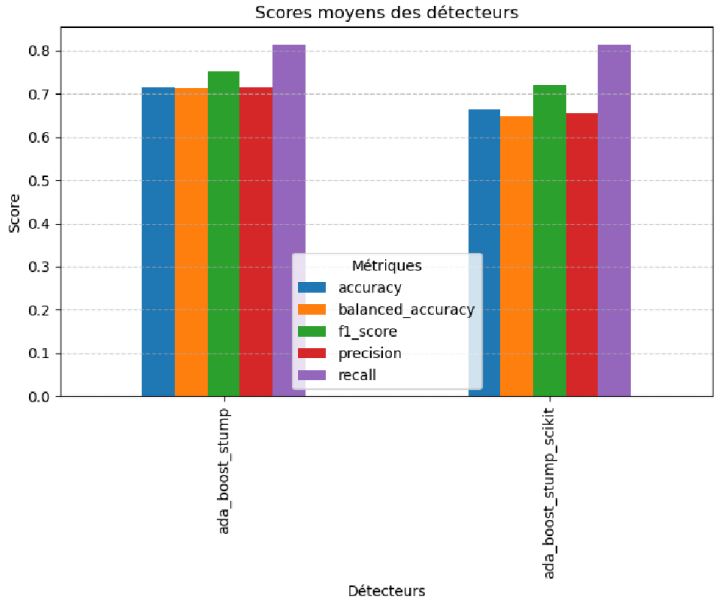
- Et si on voulait détecter le non plagiat ?



COMPARAISONS

- Un travail déjà fait
- Scikit : notre référence
- Des performances similaires à scikit

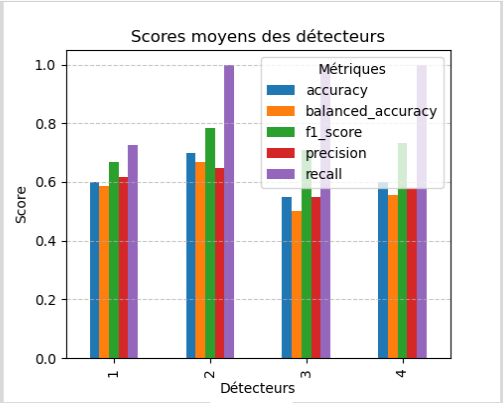




GUI

PLAGIARISM DETECTION

Classifier
Decision Tree
Featurizer
Count Vector
Criterion
Entropy
Tokenizer
Delete stopwords
Lemmatize words
Epochs
Max Depth
Learning Rate
Threshold
Patience
Tolerance
Add



Launch
Clear

4
CURRENT DETECTORS
1
PLAGIARISM DETECTION [Naive Bayes, CountVector, TOKENIZER(suppression_des_mots_vides=False, lemmatisation=False)]
2
PLAGIARISM DETECTION [LogisticRegression(learning_rate=0.01, epochs=1000, threshold=0.01, patience=5, tolerance=0.0001), CountVector, TOKENIZER(suppression_des_mots_vides=False, lemmatisation=False)]
3
PLAGIARISM DETECTION [Perceptron(learning_rate=0.01, epochs=1000, patience=0.01, tolerance=0.01), CountVector, TOKENIZER(suppression_des_mots_vides=False, lemmatisation=False)]
4
PLAGIARISM DETECTION [DecisionTree(depth=1, criterion=entropy), CountVector, TOKENIZER(suppression_des_mots_vides=False, lemmatisation=False)]

LIMITES

Tailles des données

Seulement 100 textes

Textes de longueurs modestes

Données spécifiques

Sujets scientifiques (Informatique)

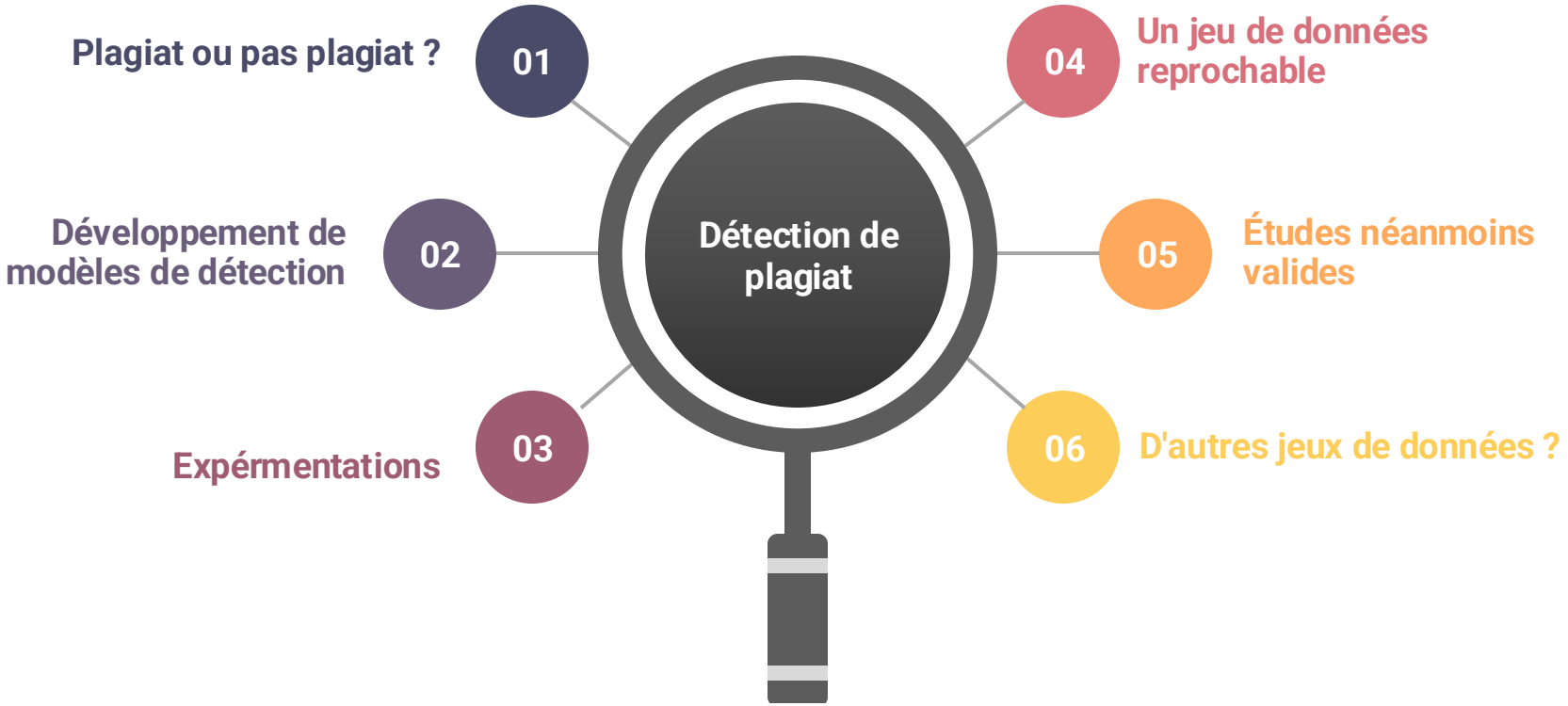
19 participants

Plagiat, vraiment ?

Qu'est réellement un plagiat ?

Cas du near copy

CONCLUSION


Plagiat ou pas plagiat ?
01
Développement de modèles de détection
02
Expérmentations
03
Détection de plagiat
04
Un jeu de données reprochable
05
Études néanmoins valides
06
D'autres jeux de données ?

Répartition des taches

Yendoumban

- Modelisation
- Pipeline
- Lemmatizer / Tokenizer
- AdaBoost
- Bagging Classifier
- Pré-analyse / WordCloud
- Expérimentations
- GUI
- Utils

Hadjer

- Tokenizer
- CountVectorizer
- TF-IDF
- Naïve Bayes
- Logistic Regression
- Perceptron
- Decision Tree
- Word Cloud
- Main tests



THANK YOU !

CHEDJARI EL MEUR Hadjer
NANMEDIGOU Yendoumban
Université de Caen Normandie
Projet 2
ADDAD Youva